AWS SERIES · LAB · DYNAMODB STREAMS + SNS + SQS

Lab: New-Employee Notifications with DynamoDB Streams, SNS and SQS

When a new employee is saved in DynamoDB, the table's stream starts a Lambda that announces it on an SNS topic. HR gets an email; IT gets a task in an SQS queue — and, optionally, a worker Lambda takes the tasks off the queue. Console, Mac and Windows steps.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Services: DynamoDB Streams · AWS Lambda · Amazon SNS · Amazon SQS · IAM · CloudWatch Logs
Before this lab: **SQS, SNS and DynamoDB Streams for Beginners**, and the lab **Lambda loads a file from S3 into DynamoDB** (keep its table, bucket and function). Region `us-east-1`. Files: `lab_streams_sns_sqs.zip`

1. What you will build
2. What happens after one upload
3. The publisher code
4. The worker code (optional part)
5. The policies
6. Before you start
7. Step 0 · Check the last lab and set variables
8. Step 1 · Turn on the DynamoDB stream
9. Step 2 · Create the SNS topic
10. Step 3 · Subscribe your email to the topic
11. Step 4 · Create the SQS queue and subscribe it to the topic
12. Step 5 · Create the IAM policy and role for the publisher
13. Step 6 · Create the publisher Lambda and connect it to the stream
14. Step 7 · Add new employees
15. Step 8 · See the chain work
16. Step 9 · (Optional) A worker Lambda empties the queue
17. Try these
18. Step 10 · Clean up
19. Troubleshooting
20. Key takeaways

1 What you will build

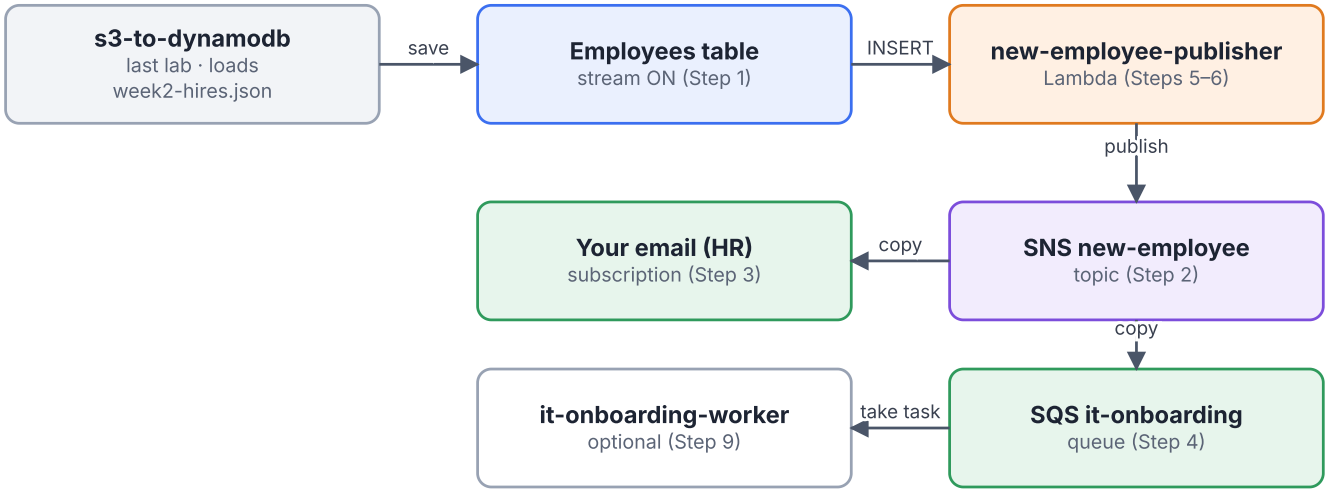


Figure 1. What you build. The grey box is from the last lab; everything else is new.

Thing	Name
DynamoDB stream	on table `Employees` (from the last lab), view type **New image**
SNS topic	`new-employee`
SQS queue	`it-onboarding`
IAM policy + role (publisher)	`new-employee-publisher-policy`, `new-employee-publisher-role`
Lambda (publisher)	`new-employee-publisher` — started by the stream
Optional: IAM policy + role (worker)	`it-onboarding-worker-policy`, `it-onboarding-worker-role`
Optional: Lambda (worker)	`it-onboarding-worker` — started by the queue

> Nothing in this lab calls another service directly. The table reports, the topic broadcasts, the queue waits — each piece only knows about the next one.

2 What happens after one upload

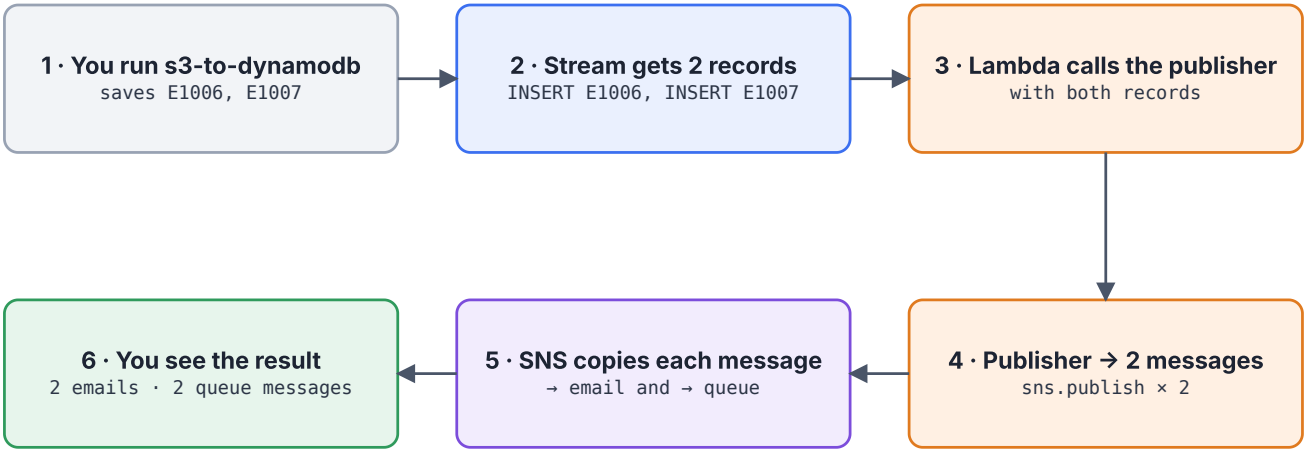


Figure 2. One upload of week2-hires.json, second by second. You only do step 1.

You load two employees with last lab's function — exactly as before. Everything after that happens by itself, within a few seconds. With the optional worker (Step 9), a 7th step follows: the worker takes both tasks off the queue and prints them.

3 The publisher code

`publisher/lambda_function.py`. The comment at the top of the file shows the same steps. The code:

```
import json
import os

import boto3

TOPIC_ARN = os.environ["TOPIC_ARN"]
sns = boto3.client("sns")


def text(item, field):
    """One text value from DynamoDB JSON: {"city": {"S": "Kolkata"}} → "Kolkata". Missing → ""."""
    return item.get(field, {}).get("S", "")


def lambda_handler(event, context):
    published = 0
    for record in event["Records"]:
        # STEP 1 — only brand-new items
        if record["eventName"] != "INSERT":
            print(f"Skipping {record['eventName']}")
            continue

        # STEP 2 — the new item, written in DynamoDB JSON ("S" = string)
        item = record["dynamodb"]["NewImage"]
        employee = {
            "emp_id": text(item, "emp_id"),
            "name": text(item, "name"),
            "department": text(item, "department"),
            "city": text(item, "city"),
        }

        # STEP 3 — one message to the topic; SNS sends it to every subscriber
        sns.publish(
            TopicArn=TOPIC_ARN,
            Subject=f"New employee: {employee['name']}",
            Message=json.dumps(employee),
        )
        print(f"Published {employee['emp_id']} {employee['name']}")
        published += 1

    return {"published": published}
```

Line	Why it is there
`for record in event["Records"]`	Lambda sends a **batch** of stream records (up to 10 here)
`if record["eventName"] != "INSERT"`	Only new employees. Edits (MODIFY) and deletes (REMOVE) are skipped
`record["dynamodb"]["NewImage"]`	The whole new item — because the stream's view type is **New image**
`text(item, "city")`	Reads `{"city": {"S": "Kolkata"}}` as `"Kolkata"`, and gives `""` if the field is missing
`sns.publish(TopicArn=…, Subject=…, Message=…)`	One message to the topic. Subject = email title; Message = our JSON

Why `text()` and not `item["city"]["S"]` ? If a stream function crashes, Lambda calls it again with the **same** records for up to 24 hours, and newer changes wait behind them. One item without a city would jam the whole chain. A stream function must not crash on bad data.

4 The worker code (optional part)

`worker/lambda_function.py` — started by the queue in Step 9:

```
import json


def lambda_handler(event, context):
    for record in event["Records"]:
        # STEP 1 — SNS wraps our message in its own JSON envelope
        envelope = json.loads(record["body"])

        # STEP 2 — our message, as published by new-employee-publisher
        employee = json.loads(envelope["Message"])

        # STEP 3 — the IT task
        print(f"IT task: prepare a laptop for {employee['name']} "
              f"({employee['emp_id']}, {employee['department']}, {employee['city']})")

    return {"tasks_done": len(event["Records"])}
```

Line	Why it is there
`json.loads(record["body"])`	The queue message body is SNS's **envelope** (JSON text)
`json.loads(envelope["Message"])`	Our employee JSON, as the publisher sent it
`return {...}` without an error	Lambda then **deletes** the batch from the queue. An error would put the messages back

5 The policies

policies/publisher-policy.json — the publisher's role (Step 5)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadTheStream",
      "Effect": "Allow",
      "Action": ["dynamodb:DescribeStream", "dynamodb:GetRecords", "dynamodb:GetShardIterator"],
      "Resource": "arn:aws:dynamodb:us-east-1:ACCOUNT_ID:table/Employees/stream/*"
    },
    {
      "Sid": "ListStreams",
      "Effect": "Allow",
      "Action": "dynamodb:ListStreams",
      "Resource": "*"
    },
    {
      "Sid": "PublishToTopic",
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:us-east-1:ACCOUNT_ID:new-employee"
    },
    {
      "Sid": "WriteLogs",
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "*"
    }
  ]
}
```

Statement	Allows
`ReadTheStream`	Reading **this table's** stream (`table/Employees/stream/*`)
`ListStreams`	Finding the stream. A list action, so `*`
`PublishToTopic`	`sns:Publish` to the topic `new-employee` only
`WriteLogs`	CloudWatch Logs

policies/queue-policy.json — on the queue itself (Step 4)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowTopicToSend",
      "Effect": "Allow",
      "Principal": { "Service": "sns.amazonaws.com" },
      "Action": "sqs:SendMessage",
      "Resource": "arn:aws:sqs:us-east-1:ACCOUNT_ID:it-onboarding",
      "Condition": { "ArnEquals": { "aws:SourceArn": "arn:aws:sns:us-east-1:ACCOUNT_ID:new-
employee" } }
    }
  ]
}
```

This is a **resource-based** policy: it is attached to the queue, has a `Principal` (SNS), and the `Condition` limits it to your topic. The console's **Subscribe to Amazon SNS topic** button writes it for you.

policies/worker-policy.json — the worker's role (Step 9)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "TakeTasksFromQueue",
      "Effect": "Allow",
      "Action": ["sqs:ReceiveMessage", "sqs:DeleteMessage", "sqs:GetQueueAttributes"],
      "Resource": "arn:aws:sqs:us-east-1:ACCOUNT_ID:it-onboarding"
    },
    {
      "Sid": "WriteLogs",
      "Effect": "Allow",
      "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "*"
    }
  ]
}
```

All three files contain `ACCOUNT_ID`. Replace it with your 12-digit account ID before pasting into the console; the command-line steps do it for you. `trust-policy.json` is the same as in the last lab.

6 Before you start

- Finish the last lab **without** its clean-up step: the table `Employees`, your bucket and the function `s3-to-dynamodb` must exist.
- Unzip `lab_streams_sns_sqs.zip` into your **Downloads** folder.
- Use an email address you can open now — you must click a confirmation link in Step 3.
- Do each step **either** in the console **or** with the commands. Commands: **PyCharm Terminal** (Mac: zsh, Windows: PowerShell), the **same window** for all steps.

> Cost: a few cents at most, and covered by the free tier. Still do Step 10 (clean up).

7 Step 0 · Check the last lab and set variables

This lab adds to the last one. The table, the bucket and the s3-to-dynamodb function must still exist.

Console

1. Region (top right): **US East (N. Virginia) us-east-1**.
2. **DynamoDB** → **Tables**: `Employees` is there (with E1001–E1005).
3. **S3**: your bucket `vs-lab-employees-<account id>` is there. **Lambda**: `s3-to-dynamodb` is there.
4. Cleaned up last time? Do Steps 0–6 of the last lab again first.

Command line — Mac

```
cd ~/Downloads/lab_streams_sns_sqs
aws configure set region us-east-1

ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
BUCKET="vs-lab-employees-$ACCOUNT"
EMAIL="you@example.com"          # ← your real email address

aws dynamodb scan --table-name Employees --select COUNT
aws lambda get-function --function-name s3-to-dynamodb --query Configuration.FunctionName --output
text
```

Command line — Windows (PowerShell)

```
cd ~\Downloads\lab_streams_sns_sqs
aws configure set region us-east-1

$ACCOUNT = aws sts get-caller-identity --query Account --output text
$BUCKET = "vs-lab-employees-$ACCOUNT"
$EMAIL = "you@example.com"        # ← your real email address

aws dynamodb scan --table-name Employees --select COUNT
aws lambda get-function --function-name s3-to-dynamodb --query Configuration.FunctionName --output
text
```

> **You should see:** `"Count": 5` (or 3 if you skipped Step 8 last time) and `s3-to-dynamodb` .

8 Step 1 · Turn on the DynamoDB stream

From now on, every change to the table is written to its stream.

Console

1. **DynamoDB** → **Tables** → `Employees` → **Exports and streams** tab.
2. Under **DynamoDB stream details** → **Turn on**.
3. View type: **New image** (the whole item after the change) → **Turn on stream**.

Command line — Mac

```
aws dynamodb update-table --table-name Employees --stream-specification
"StreamEnabled=true,StreamViewType=NEW_IMAGE"

STREAM_ARN=$(aws dynamodb describe-table --table-name Employees --query Table.LatestStreamArn --
output text)
echo $STREAM_ARN
```

Command line — Windows (PowerShell)

```
aws dynamodb update-table --table-name Employees --stream-specification
"StreamEnabled=true,StreamViewType=NEW_IMAGE"

$STREAM_ARN = aws dynamodb describe-table --table-name Employees --query Table.LatestStreamArn --
output text
echo $STREAM_ARN
```

You should see: A stream ARN like `arn:aws:dynamodb:us-east-1:<account id>:table/Employees/stream/2026-11-05T…` . Items already in the table are **not** in the stream — only changes from now on.

9 Step 2 · Create the SNS topic

The topic is the loudspeaker: one message in, a copy out to every subscriber.

Console

1. **SNS** → **Topics** → **Create topic**.
2. Type: **Standard**. Name: `new-employee` → **Create topic**.
3. Copy the topic **ARN** from the topic page — you need it in Step 6.

Command line — Mac

```
TOPIC_ARN=$(aws sns create-topic --name new-employee --query TopicArn --output text)
echo $TOPIC_ARN
```

Command line — Windows (PowerShell)

```
$TOPIC_ARN = aws sns create-topic --name new-employee --query TopicArn --output text
echo $TOPIC_ARN
```

You should see: `arn:aws:sns:us-east-1:<account id>:new-employee`

10 Step 3 · Subscribe your email to the topic

HR's inbox (yours, in the lab) becomes the first subscriber.

Console

1. Open the topic → **Create subscription**.

2. Protocol: **Email**. Endpoint: your email address → **Create subscription**.
3. Open your inbox: email **AWS Notification - Subscription Confirmation** → click **Confirm subscription**.
4. Back on the topic page, the subscription status changes from **Pending confirmation** to **Confirmed**.

Command line — Mac

```
aws sns subscribe --topic-arn $TOPIC_ARN --protocol email --notification-endpoint $EMAIL

# open the email and click "Confirm subscription", then:
aws sns list-subscriptions-by-topic --topic-arn $TOPIC_ARN --query "Subscriptions[].
[Protocol,Endpoint,SubscriptionArn]" --output table
```

Command line — Windows (PowerShell)

```
aws sns subscribe --topic-arn $TOPIC_ARN --protocol email --notification-endpoint $EMAIL

# open the email and click "Confirm subscription", then:
aws sns list-subscriptions-by-topic --topic-arn $TOPIC_ARN --query "Subscriptions[].
[Protocol,Endpoint,SubscriptionArn]" --output table
```

You should see: One `email` subscription with a long subscription ARN. `PendingConfirmation` means you haven't clicked the link yet (check spam).

11 Step 4 · Create the SQS queue and subscribe it to the topic

IT's to-do list. The queue also needs a policy that lets this topic put messages in it.

Console

1. **SQS** → **Create queue**. Type: **Standard**. Name: `it-onboarding`. Keep all defaults → **Create queue**.
2. On the queue page → **Subscribe to Amazon SNS topic** → pick `new-employee` → **Save**.
3. Open the queue's **Access policy** tab: the console added a statement that allows `sns.amazonaws.com` to `sqs:SendMessage`, only for your topic. Compare it with `policies/queue-policy.json`.

Command line — Mac

```
QUEUE_URL=$(aws sqs create-queue --queue-name it-onboarding --query QueueUrl --output text)
QUEUE_ARN=$(aws sqs get-queue-attributes --queue-url $QUEUE_URL --attribute-names QueueArn --query
Attributes.QueueArn --output text)

sed "s/ACCOUNT_ID/$ACCOUNT/g" policies/queue-attributes.json > my-queue-attributes.json
aws sqs set-queue-attributes --queue-url $QUEUE_URL --attributes file://my-queue-attributes.json

aws sns subscribe --topic-arn $TOPIC_ARN --protocol sqs --notification-endpoint $QUEUE_ARN
```

Command line — Windows (PowerShell)

```
$QUEUE_URL = aws sqs create-queue --queue-name it-onboarding --query QueueUrl --output text
$QUEUE_ARN = aws sqs get-queue-attributes --queue-url $QUEUE_URL --attribute-names QueueArn --query
Attributes.QueueArn --output text

(Get-Content policies\queue-attributes.json) -replace "ACCOUNT_ID", $ACCOUNT | Set-Content -
Encoding ascii my-queue-attributes.json
aws sqs set-queue-attributes --queue-url $QUEUE_URL --attributes file://my-queue-attributes.json

aws sns subscribe --topic-arn $TOPIC_ARN --protocol sqs --notification-endpoint $QUEUE_ARN
```

You should see: The topic now has two subscriptions: your email and the queue. A queue subscription needs no confirmation.

`queue-attributes.json` holds the same policy as `queue-policy.json`, written as one text value — that is the shape `set-queue-attributes` expects.

12 Step 5 · Create the IAM policy and role for the publisher

The publisher Lambda must read the stream, publish to the topic and write logs — nothing more.

Console

1. Open `policies/publisher-policy.json` in a text editor; replace both `ACCOUNT_ID` with your account ID.
2. **IAM** → **Policies** → **Create policy** → **JSON** → paste → **Next** → name `new-employee-publisher-policy` → **Create policy**.
3. **IAM** → **Roles** → **Create role** → **AWS service** → **Lambda** → **Next** → tick `new-employee-publisher-policy` → **Next**.
4. Role name: `new-employee-publisher-role` → **Create role**.

Command line — Mac

```
sed "s/ACCOUNT_ID/$ACCOUNT/g" policies/publisher-policy.json > my-publisher-policy.json
PUB_POLICY_ARN=$(aws iam create-policy --policy-name new-employee-publisher-policy --policy-
document file://my-publisher-policy.json --query Policy.Arn --output text)

PUB_ROLE_ARN=$(aws iam create-role --role-name new-employee-publisher-role --assume-role-policy-
document file://policies/trust-policy.json --query Role.Arn --output text)
aws iam attach-role-policy --role-name new-employee-publisher-role --policy-arn $PUB_POLICY_ARN
echo $PUB_ROLE_ARN
```

Command line — Windows (PowerShell)

```
(Get-Content policies\publisher-policy.json) -replace "ACCOUNT_ID", $ACCOUNT | Set-Content -
Encoding ascii my-publisher-policy.json
$PUB_POLICY_ARN = aws iam create-policy --policy-name new-employee-publisher-policy --policy-
document file://my-publisher-policy.json --query Policy.Arn --output text

$PUB_ROLE_ARN = aws iam create-role --role-name new-employee-publisher-role --assume-role-policy-
document file://policies/trust-policy.json --query Role.Arn --output text
aws iam attach-role-policy --role-name new-employee-publisher-role --policy-arn $PUB_POLICY_ARN
echo $PUB_ROLE_ARN
```

You should see: `arn:aws:iam::<account id>:role/new-employee-publisher-role`

13 Step 6 · Create the publisher Lambda and connect it to the stream

The stream becomes the function's trigger: Lambda reads the stream and calls the function with new records.

Console

1. **Lambda** → **Create function** → **Author from scratch**. Name `new-employee-publisher`, Runtime **Python 3.14**.
2. **Change default execution role** → **Use an existing role** → `new-employee-publisher-role` → **Create function**.
3. **Code** tab: paste `publisher/lambda_function.py` → **Deploy**.
4. **Configuration** → **Environment variables** → **Edit** → add `TOPIC_ARN` = your topic ARN (Step 2) → **Save**.
5. **Add trigger** (top of the page) → source **DynamoDB** → table `Employees` → Batch size `10` → Starting position **Latest** → **Add**.
6. **Configuration** → **Triggers**: wait until the trigger state is **Enabled** (about a minute).

Command line — Mac

```
cd publisher
zip ../publisher.zip lambda_function.py
cd ..
sleep 10

aws lambda create-function --function-name new-employee-publisher --runtime python3.14 --handler
lambda_function.lambda_handler --role $PUB_ROLE_ARN --zip-file fileb://publisher.zip --timeout 30 -
-environment "Variables={TOPIC_ARN=$TOPIC_ARN}"
aws lambda wait function-active-v2 --function-name new-employee-publisher

aws lambda create-event-source-mapping --function-name new-employee-publisher --event-source-arn
$STREAM_ARN --starting-position LATEST --batch-size 10
aws lambda list-event-source-mappings --function-name new-employee-publisher --query
"EventSourceMappings[].State" --output text
```

Command line — Windows (PowerShell)

```
cd publisher
Compress-Archive -Path lambda_function.py -DestinationPath ..\publisher.zip -Force
cd ..
Start-Sleep -Seconds 10

aws lambda create-function --function-name new-employee-publisher --runtime python3.14 --handler
lambda_function.lambda_handler --role $PUB_ROLE_ARN --zip-file fileb://publisher.zip --timeout 30 -
-environment "Variables={TOPIC_ARN=$TOPIC_ARN}"
aws lambda wait function-active-v2 --function-name new-employee-publisher

aws lambda create-event-source-mapping --function-name new-employee-publisher --event-source-arn
$STREAM_ARN --starting-position LATEST --batch-size 10
aws lambda list-event-source-mappings --function-name new-employee-publisher --query
"EventSourceMappings[].State" --output text
```

You should see: `Enabled` . If it says `Creating` or `Enabling` , wait a minute and run the last command again.

Wait for **Enabled** before Step 7. **Latest** means "start reading from now" — changes made while the trigger is still being set up are not sent.

14 Step 7 · Add new employees

Use the last lab's function to load week2-hires.json. You never call the publisher yourself.

Console

1. **S3** → your bucket → **Upload** `data/week2-hires.json` (from this lab's folder).
2. **Lambda** → `s3-to-dynamodb` → **Test** tab → change `"key"` to `"week2-hires.json"` → **Test**.

Command line — Mac

```
aws s3 cp data/week2-hires.json "s3://$BUCKET/week2-hires.json"

cat > test-event.json <<EOF
{ "bucket": "$BUCKET", "key": "week2-hires.json" }
EOF
aws lambda invoke --function-name s3-to-dynamodb --cli-binary-format raw-in-base64-out --payload
file://test-event.json response.json
cat response.json
```

Command line — Windows (PowerShell)

```
aws s3 cp data\week2-hires.json "s3://$BUCKET/week2-hires.json"

@"
{ "bucket": "$BUCKET", "key": "week2-hires.json" }
"@ | Set-Content -Encoding ascii test-event.json
aws lambda invoke --function-name s3-to-dynamodb --cli-binary-format raw-in-base64-out --payload
file://test-event.json response.json
Get-Content response.json
```

You should see: `{"file": "week2-hires.json", "items_written": 2}` — E1006 Arjun and E1007 Divya are new, so the stream gets two INSERT records.

15 Step 8 · See the chain work

Within a few seconds: two emails, two messages waiting in the queue, and the publisher's log.

Console

1. **Your inbox**: two emails, **New employee: Arjun** and **New employee: Divya**.
2. **SQS** → `it-onboarding` → **Send and receive messages** → **Poll for messages**: 2 messages. Click one → **Body**: SNS's envelope, with your employee JSON inside `"Message"`.
3. Poll again after 30 seconds: the same 2 messages come back — nobody deleted them yet.
4. **Lambda** → `new-employee-publisher` → **Monitor** → **View CloudWatch logs**: `Published E1006 Arjun`, `Published E1007 Divya`.

Command line — Mac

```
aws logs tail /aws/lambda/new-employee-publisher --since 10m

aws sqs receive-message --queue-url $QUEUE_URL --max-number-of-messages 10 --wait-time-seconds 10 --query "Messages[].Body" --output text
```

Command line — Windows (PowerShell)

```
aws logs tail /aws/lambda/new-employee-publisher --since 10m

aws sqs receive-message --queue-url $QUEUE_URL --max-number-of-messages 10 --wait-time-seconds 10 --query "Messages[].Body" --output text
```

You should see: Two emails, two queue messages, two `Published …` log lines.

A received message is hidden for 30 seconds (the **visibility timeout**), then reappears, because nobody deleted it. That is how SQS makes sure no task is lost. The worker in Step 9 deletes them after doing the work.

16 Step 9 · (Optional) A worker Lambda empties the queue

Instead of polling by hand, Lambda reads the queue, runs the worker, and deletes each message after success.

Console

1. Replace `ACCOUNT_ID` in `policies/worker-policy.json`. **IAM** → create policy `it-onboarding-worker-policy` from it.
2. **IAM** → create role `it-onboarding-worker-role` (AWS service → Lambda) with that policy.
3. **Lambda** → create `it-onboarding-worker`, Python 3.14, existing role `it-onboarding-worker-role`. Paste `worker/lambda_function.py` → **Deploy**.

4. **Add trigger** → **SQS** → queue `it-onboarding` → Batch size `10` → **Add**.
5. Within a minute: **Monitor** → logs show `IT task: prepare a laptop for Arjun …` and `… for Divya …`. In SQS, **Messages available** drops to 0.

Command line — Mac

```
sed "s/ACCOUNT_ID/$ACCOUNT/g" policies/worker-policy.json > my-worker-policy.json
WRK_POLICY_ARN=$(aws iam create-policy --policy-name it-onboarding-worker-policy --policy-document
file://my-worker-policy.json --query Policy.Arn --output text)
WRK_ROLE_ARN=$(aws iam create-role --role-name it-onboarding-worker-role --assume-role-policy-
document file://policies/trust-policy.json --query Role.Arn --output text)
aws iam attach-role-policy --role-name it-onboarding-worker-role --policy-arn $WRK_POLICY_ARN

cd worker
zip ../worker.zip lambda_function.py
cd ..
sleep 10

aws lambda create-function --function-name it-onboarding-worker --runtime python3.14 --handler
lambda_function.lambda_handler --role $WRK_ROLE_ARN --zip-file fileb://worker.zip --timeout 30
aws lambda wait function-active-v2 --function-name it-onboarding-worker
aws lambda create-event-source-mapping --function-name it-onboarding-worker --event-source-arn
$QUEUE_ARN --batch-size 10

sleep 60
aws logs tail /aws/lambda/it-onboarding-worker --since 10m
```

Command line — Windows (PowerShell)

```
(Get-Content policies\worker-policy.json) -replace "ACCOUNT_ID", $ACCOUNT | Set-Content -Encoding
ascii my-worker-policy.json
$WRK_POLICY_ARN = aws iam create-policy --policy-name it-onboarding-worker-policy --policy-document
file://my-worker-policy.json --query Policy.Arn --output text
$WRK_ROLE_ARN = aws iam create-role --role-name it-onboarding-worker-role --assume-role-policy-
document file://policies/trust-policy.json --query Role.Arn --output text
aws iam attach-role-policy --role-name it-onboarding-worker-role --policy-arn $WRK_POLICY_ARN

cd worker
Compress-Archive -Path lambda_function.py -DestinationPath ..\worker.zip -Force
cd ..
Start-Sleep -Seconds 10

aws lambda create-function --function-name it-onboarding-worker --runtime python3.14 --handler
lambda_function.lambda_handler --role $WRK_ROLE_ARN --zip-file fileb://worker.zip --timeout 30
aws lambda wait function-active-v2 --function-name it-onboarding-worker
aws lambda create-event-source-mapping --function-name it-onboarding-worker --event-source-arn
$QUEUE_ARN --batch-size 10

Start-Sleep -Seconds 60
aws logs tail /aws/lambda/it-onboarding-worker --since 10m
```

You should see: `IT task: prepare a laptop for Arjun (E1006, Finance, Kolkata)` and the same for Divya. The queue is empty.

17 Try these

Four small experiments in the console. Each one shows one idea from the guide.

Do this	What you see	Why
Run `s3-to-dynamodb` again with `week2-hires.json`	No new emails, no new queue messages	The items are exactly the same, so nothing changed — the stream writes no record
DynamoDB → **Explore table items** → open E1006 → change `city` to `Mumbai` → **Save**	No email. Publisher log: `Skipping MODIFY`	The stream sent a MODIFY record; the code acts only on INSERT
DynamoDB → **Create item**: `emp_id` E1008, plus `name`, `department`, `city` (all String)	An email for E1008 and an IT task	Any new item triggers the chain — not only file uploads
SNS → **Subscriptions** → delete your email subscription. Then create item E1009	No email, but the IT task still arrives	Each subscriber is independent: removing one doesn't affect the others

> If you skipped the worker (Step 9), the queue keeps every task. Poll it to see them pile up — nothing is lost while nobody works on them.

18 Step 10 · Clean up

Remove this lab's pieces. Skip the worker lines if you didn't do Step 9. New terminal? Run Step 0 again first.

Console

1. **Lambda** → each function (`new-employee-publisher`, `it-onboarding-worker`) → **Configuration** → **Triggers** → delete the trigger; then **Actions** → **Delete** the function.
2. **CloudWatch** → **Log groups** → delete `/aws/lambda/new-employee-publisher` and `/aws/lambda/it-onboarding-worker`.
3. **IAM** → delete both roles, then both policies (`new-employee-publisher-…`, `it-onboarding-worker-…`).
4. **SNS** → delete topic `new-employee` (its subscriptions go with it). **SQS** → delete queue `it-onboarding`.
5. **DynamoDB** → `Employees` → **Exports and streams** → **Turn off** the stream.
6. Finished with the series? Do the last lab's Step 9 (clean up) too.

Command line — Mac

```
UUID=$(aws lambda list-event-source-mappings --function-name new-employee-publisher --query
"EventSourceMappings[0].UUID" --output text)
aws lambda delete-event-source-mapping --uuid $UUID
UUID=$(aws lambda list-event-source-mappings --function-name it-onboarding-worker --query
"EventSourceMappings[0].UUID" --output text)
aws lambda delete-event-source-mapping --uuid $UUID

aws lambda delete-function --function-name new-employee-publisher
aws lambda delete-function --function-name it-onboarding-worker
aws logs delete-log-group --log-group-name /aws/lambda/new-employee-publisher
aws logs delete-log-group --log-group-name /aws/lambda/it-onboarding-worker

aws iam detach-role-policy --role-name new-employee-publisher-role --policy-arn
"arn:aws:iam::${ACCOUNT}:policy/new-employee-publisher-policy"
aws iam delete-role --role-name new-employee-publisher-role
aws iam delete-policy --policy-arn "arn:aws:iam::${ACCOUNT}:policy/new-employee-publisher-policy"
aws iam detach-role-policy --role-name it-onboarding-worker-role --policy-arn
"arn:aws:iam::${ACCOUNT}:policy/it-onboarding-worker-policy"
aws iam delete-role --role-name it-onboarding-worker-role
aws iam delete-policy --policy-arn "arn:aws:iam::${ACCOUNT}:policy/it-onboarding-worker-policy"

aws sns delete-topic --topic-arn "arn:aws:sns:us-east-1:${ACCOUNT}:new-employee"
aws sqs delete-queue --queue-url $(aws sqs get-queue-url --queue-name it-onboarding --query
QueueUrl --output text)
aws dynamodb update-table --table-name Employees --stream-specification StreamEnabled=false
```

Command line — Windows (PowerShell)

```
$UUID = aws lambda list-event-source-mappings --function-name new-employee-publisher --query
"EventSourceMappings[0].UUID" --output text
aws lambda delete-event-source-mapping --uuid $UUID
$UUID = aws lambda list-event-source-mappings --function-name it-onboarding-worker --query
"EventSourceMappings[0].UUID" --output text
aws lambda delete-event-source-mapping --uuid $UUID

aws lambda delete-function --function-name new-employee-publisher
aws lambda delete-function --function-name it-onboarding-worker
aws logs delete-log-group --log-group-name /aws/lambda/new-employee-publisher
aws logs delete-log-group --log-group-name /aws/lambda/it-onboarding-worker

aws iam detach-role-policy --role-name new-employee-publisher-role --policy-arn
"arn:aws:iam::${ACCOUNT}:policy/new-employee-publisher-policy"
aws iam delete-role --role-name new-employee-publisher-role
aws iam delete-policy --policy-arn "arn:aws:iam::${ACCOUNT}:policy/new-employee-publisher-policy"
aws iam detach-role-policy --role-name it-onboarding-worker-role --policy-arn
"arn:aws:iam::${ACCOUNT}:policy/it-onboarding-worker-policy"
aws iam delete-role --role-name it-onboarding-worker-role
aws iam delete-policy --policy-arn "arn:aws:iam::${ACCOUNT}:policy/it-onboarding-worker-policy"

aws sns delete-topic --topic-arn "arn:aws:sns:us-east-1:${ACCOUNT}:new-employee"
$QUEUE_URL = aws sqs get-queue-url --queue-name it-onboarding --query QueueUrl --output text
aws sqs delete-queue --queue-url $QUEUE_URL
aws dynamodb update-table --table-name Employees --stream-specification StreamEnabled=false
```

You should see: Only the last lab's table, bucket and function remain (delete them with that lab's Step 9 when you're done).

19 Troubleshooting

What you see	Cause	Fix
No email at all	Email subscription not confirmed, or in spam	SNS → Subscriptions: status must be **Confirmed**
Nothing happens, publisher has no logs	Trigger not **Enabled** yet, stream off, or the data didn't change	Lambda → Configuration → Triggers; DynamoDB → Exports and streams; use **new** employees
Trigger won't save: role can't read the stream	Publisher role is missing the stream permissions	Check `ReadTheStream`, `ListStreams` and the account ID (Step 5)
Publisher log: `AuthorizationError … sns:Publish`	Wrong topic ARN in the policy or in `TOPIC_ARN`	Compare both with the topic page
`KeyError: 'TOPIC_ARN'`	Environment variable missing	Step 6: add `TOPIC_ARN`
Emails arrive, queue stays empty	Queue not subscribed, or its access policy is missing	SNS topic: 2 subscriptions? SQS → Access policy (Step 4)
Worker trigger won't save: no permission for ReceiveMessage	Worker role policy missing or wrong queue ARN	Check `worker-policy.json` (Step 9)
Queue messages keep coming back	The worker fails, so Lambda doesn't delete them	Worker → Monitor → logs show the error
Old employees E1001–E1005 never sent	Expected — the stream only has changes after it was turned on	—

20 Key takeaways

1. Turning on a stream makes the table report every change; a Lambda trigger reacts to it — you never call the publisher.
2. The publisher only publishes. SNS makes a copy for every subscriber: email for HR, a queue for IT.
3. The queue keeps IT's tasks until a worker deletes them; a failed worker means the message comes back.
4. Stream functions must not crash on bad data — a failed batch is retried and blocks newer changes.
5. Two kinds of permission: roles for the Lambdas, and an access policy on the queue that lets the topic send.
6. Adding a new team later = one more subscription. No code changes.